Caso de estudio N°1: Auditoría de un incidente en SistemaSoporte

Curso: SC-701 Programación Avanzada en Web | **Semana 7** | **Valor:** 15% de la nota final **Modalidad:** individual, en clase, supervisado | **Tiempo:** 60 minutos | **Versión asignada:** B

Reglas

- La actividad es **individual** y se hace en clase. No se permite hablar con compañeros.
- **Sin inteligencia artificial y sin celular. Sin conexión a internet:** después de descargar este archivo, **apague el Wi-Fi o active el modo avión** y manténgalo así hasta el final. El profesor lo verificará.
- Material permitido: la presentación de la Semana 6 y el código del proyecto de clase, **que ya deben tener descargados**.
- Se califica **únicamente la versión que le fue asignada**.
- Entrega: un archivo Word o PDF llamado `Caso1_Carnet_Apellido` en el espacio **Caso de estudio N°1** de la plataforma, antes de que termine el tiempo. No hay prórroga. Solo en ese momento, con el profesor presente, puede volver a conectarse para subirlo.

Escenario

La empresa **SoporteYa S.A.** puso en producción hace una semana la API de **SistemaSoporte**, la misma arquitectura en capas que vimos en clase (`API`, `Flujo`, `DA`, `Abtracciones`), con Entity Framework Core y SQL Server en Azure. Durante la semana el equipo de soporte registró tres incidentes.

Usted es la persona contratada para **auditar el código**. Para cada incidente tiene el reporte y el fragmento de código donde se origina el problema. Su trabajo es explicar qué pasa, por qué pasa y qué se puede hacer, como si le entregara el informe a la gerencia técnica de la empresa.

La entidad que usan todos los fragmentos es esta:

```
// Abtracciones/Modelos/Cliente.cs
namespace Abtracciones.Modelos
{
    public class Cliente
    {
        public int Id { get; set; }
        public string Nombre { get; set; }
        public string Correo { get; set; }
        public string Telefono { get; set; }
        public string? Empresa { get; set; }
    }
}
```

Qué debe entregar

Para **cada uno de los 3 hallazgos** de su versión, responda estas cuatro partes, con ese orden y esos títulos:

1. **Qué pasa:** explique la causa en el código. No basta con repetir el síntoma del reporte.
2. **Fundamento:** el concepto visto en clase que explica el problema (con sus palabras, no copiado).
3. **Alternativas de solución:** al menos **dos** alternativas distintas, con una ventaja y una desventaja de cada una.
4. **Recomendación:** cuál alternativa elegiría y por qué.

Al final, una sección de **Priorización**:

1. Ordene sus 3 hallazgos del **más urgente al menos urgente** y explique el criterio que usó. Indique también qué consecuencia tendría para la empresa no resolver el primero.

Puede incluir pequeños fragmentos de código corregido, pero no es obligatorio. Se valora más el análisis que el código.

VERSIÓN B

Hallazgo 1. Reporte INC-201

"El repositorio público de GitHub del proyecto muestra la contraseña de la base de datos. Se detectaron bases de datos nuevas en el servidor que nadie del equipo creó."

```
// API/appsettings.json (versionado en Git desde el primer commit)
{
  "ConnectionStrings": {
    "SistemaSoporte": "Server=tcp:soporteya-sql.database.windows.net,1433;Initial
Catalog=SistemaSoporteDB;User ID=adminsoporte;Password=Soporte2026!;Encrypt=True;"
  },
  "Logging": {
    "LogLevel": { "Default": "Information" }
  }
}
```

Hallazgo 2. Reporte INC-202

"Al escribir en el buscador de correos el texto `'; DROP TABLE Clientes; --`, la API respondió con un error 500. Al revisar el ambiente de pruebas, la tabla `Clientes` había desaparecido."

```
// DA/ClienteDA.cs
public async Task<List<Cliente>> BuscarPorCorreo(string correo)
{
    // Usa el procedimiento almacenado de búsqueda por correo.
    var consulta = $"EXEC dbo.sp_BuscarClientesPorCorreo '{correo}'";
    return await _contexto.Clientes.FromSqlRaw(consulta).ToListAsync();
}
```

Hallazgo 3. Reporte INC-203

"El endpoint `GET /api/clientes` tarda cerca de 9 segundos en responder y el consumo de memoria del servidor sube mucho. La tabla ya tiene 80 000 clientes."

```
// DA/ClienteDA.cs
public async Task<List<Cliente>> Obtener()
{
    // Devuelve todos los clientes registrados.
    var clientes = await _contexto.Clientes.ToListAsync();
    return clientes;
}

// API/Controllers/ClientesController.cs
[HttpGet]
public async Task<IActionResult> Obtener() => Ok(await _flujo.Obtener());
```

Cómo se califica (100 puntos)

Se aplica la rúbrica oficial del programa de casos prácticos: presentar el caso a tiempo, plantear alternativas de solución claras, demostrar dominio del tema, un reporte ordenado con marco teórico, y relacionar lo visto en clase con situaciones reales.

Criterio	Puntos
Cada hallazgo (3 hallazgos de 25 puntos): qué pasa (8), fundamento (6), alternativas con ventajas y desventajas (7), recomendación justificada (4)	75
Priorización con criterio e impacto para la empresa	15
Claridad, orden del reporte y entrega a tiempo	10
Total	**100**